Valid Vault Enterprise Phase 1 Build Plan

Nov 12, 2026 · Michael Carter

Phase 1 adds shared company vaults to the shipping extension and Android app. It is built in nine stages, starting with a test of whether the extension can run a shared folder at all, and each stage closes only when its exit tests pass. The estimate is about 36 weeks for one full-time engineer, including 20% contingency, and it gets re-estimated at the end of stage 0.

1. Scope

In phase 1:

- **Companies:** an admin creates the company on their desktop, which makes its signing key and its recovery key
- **Company settings file:** the rules and public keys in one signed file, pushed through Chrome policy or Android managed configuration, or imported by hand
- **Member devices:** each device gets its own key pairs, locked under its existing unlock, and shows a device code for the admin to approve
- **Shared vaults:** a vault key wrapped once for each member device, stored as a folder on a file server or NAS
- **Removal with re-keying:** removing a device gives the vault a new key, wrapped only for the devices that remain
- **Executive approval:** the company picks which actions need several executives to sign
- **Split recovery key:** the recovery key is split into pieces, such as 3 of 5, kept offline by the holders
- **Phones:** receive shared vaults from a member desktop by QR, read and fill only

Not in phase 1:

- IT-managed devices with no master password, and fill-only members (phase 2)
- Audit logs, reports, and the departure checklist built from them (phase 3)
- Editing shared vaults on phones
- The local vault service (its own repo, later) and custom builds
- Directory sync and single sign-on

2. Baseline in 0.9.4.7

Checked against the repo at 0.9.4.7. Phase 1 builds on these modules and changes as little of them as it can.

Module	What it does today	Phase 1 use
`crypto.js`	AES-256-GCM with a context label bound as authenticated data (prefix `valid-vault/`), PBKDF2-SHA256 at 600,000 rounds, HKDF-SHA256 for the fingerprint PRF output. No ECDH or ECDSA yet	Context labels and the encrypt and decrypt helpers reused. Public-key work goes in a new module
`keypackage.js`	Key backup file: passphrase plus Compound Questions, 1,000,000 rounds, versioned format	Same format protects the admin's backup of the company signing key
`auth.js`, `session.js`, `background.js`	Unlock methods, master key wrapping, session key held only in the background	Device private keys unwrap with the session. Shared vault keys live in `background.js` too
`linkedvault.js`	Uni-Vault: one linked file picked through `showSaveFilePicker` or `showOpenFilePicker`, handle kept in IndexedDB, permission re-checked, whole file rewritten with `createWritable`, changes spotted by modified time and size	Starting point only. Shared folders need a directory handle, listing, and deletion, which no shipping code does yet. Stage 0 proves them
`vaultfile.js`, `store.js`	Versioned bundle format with schema checks, verify and roll back on upgrade, partial-sync recovery, a sync journal, and a Web Lock per browser profile	Format checks, the journal, and the verify-and-roll-back pattern. The Web Lock does not span machines

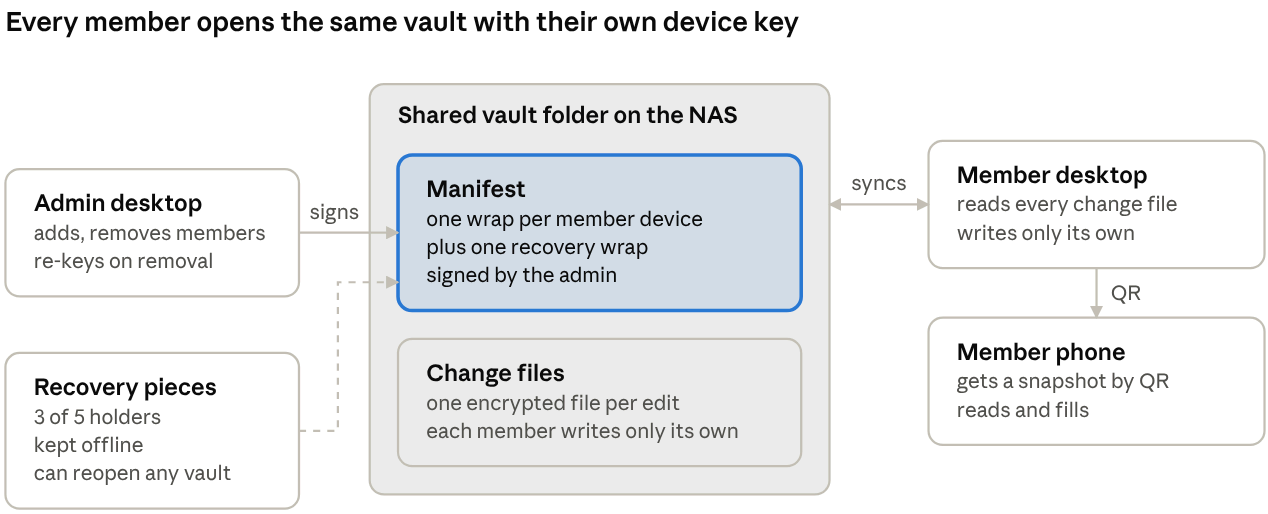
Module	What it does today	Phase 1 use
Personal merge	Timestamps with deletion markers. A delete and an edit in the same millisecond resolve to the delete (`merge_test`)	Shared vaults use edit counters instead (section 8)
`fountain.js`, `qrcode.js`, `jsqr.js`, `scanworker.js`	QR fountain transfer and the vendored decoder	Phone delivery, device codes, recovery pieces
`bip39-words.js`	BIP-39 word list	Device code check words
`manage.js`, `manage.html`	Manage page and its tabs	New Organization tab
`fileimport.js`, `importflow.js`	Imports from other password managers	Moving a company's existing logins into shared vaults
`tests/`	`node --test` with a Playwright extension harness and a phone harness: 29 files, 193 tests, plus 27 Android unit tests	Every stage adds to it
`manifest.json`	Extension pages run with `connect-src 'none'`	Unchanged

3. Rules it keeps

- **Verify by unwrap.** A device proves access by unwrapping the vault key. Nothing stores a password or key hash to compare against
- **One key, wrapped per holder.** The vault key is wrapped once per member device and once for recovery, the same pattern as the master key's unlock methods
- **The session key stays in background.js.** Shared vault keys live there too. Content scripts only receive the one value they asked to fill
- **No service connections.** Shared folders are reached as files through Chrome's File System Access API. `connect-src 'none'` stays, and there is no Valid Vault service to contact
- **No new dependencies.** ECDH, ECDSA, and HKDF come from WebCrypto. The recovery split is written in-house

- **Unknown formats throw.** Every file carries a format name and version, and an unrecognized version stops with an error
- **Risky changes follow the migration template.** Re-keying writes an encrypted journal first, checks what it wrote against a fingerprint kept in the journal, and rolls back on any mismatch
- **Personal vaults stay separate.** They are never shared and never recovered by the company. Company settings can only turn them off on company laptops or set where they are stored

4. How a shared vault sits on the network



shared vault on the company network · admin, members, recovery

The folder holds only encrypted data and the admin-signed manifest, so a copy is useless without a member's device key. Members never change the manifest, and each writes only its own change files, so two people saving at once never overwrite each other.

New modules. The names are proposals.

Module	Job	Stage
`sharedfolder.js`	Storage adapter over a directory handle: list, read, write, delete	0, finished in 4
`orgcrypto.js`	Canonical encoding, ECDH key wrapping, ECDSA signing, public key checks	1

Module	Job	Stage
`recoverysplit.js`	SLIP-0039 split and combine	1
`orgconfig.js`	Company settings: build, sign, verify, pin, enforce	2
`devicekeys.js`	Device key pairs, device codes, enrollment	3
`sharedvault.js`	Manifest, change files, validation, merge	4
`rekey.js`	Re-key state machine and journal	5
Organization tab in `manage.js`	Admin and employee screens	6

5. File formats

Encoding. Every signed or fingerprinted structure is canonical JSON: UTF-8, keys sorted, no whitespace, integers only, binary as unpadded base64url. A signature covers the canonical bytes of the object without its `sig` field.

Folder layout:

```
<share>\ValidVault\<orgId>\
  settings.vvs                          company settings, signed
  vaults\<vaultId>\
    manifest.vvm                        members and wrapped keys, signed
    gen-<n>\snapshot.vvx                full encrypted snapshot for key
generation n
    gen-<n>\changes\<deviceId>\<seq>.vvc   one folder per writing device
```

Each device writes only inside its own `changes` folder and never rewrites a file it already wrote, so no two machines ever write the same file.

Manifest (`valid-vault-manifest` , version 1):

Field	Contents	Readable without a key
`orgId` , `vaultId`	Random 128-bit ids	Yes

Field	Contents	Readable without a key
`epoch`	Integer, up by 1 on every change	Yes
`generation`	Key generation, up by 1 on every re-key	Yes
`members`	Per device: `deviceId`, ECDH public key, ECDSA public key, role (`reader` or `writer`). No names	Yes
`wraps`	Per device: ephemeral public key, IV, wrapped vault key	Ciphertext only
`recoveryWrap`	The vault key wrapped to the recovery public key	Ciphertext only
`approvals`	Executive signatures, when the action needs them	Yes
`sig`	Company ECDSA signature	Yes

Device names live only in the admin's own encrypted vault.

Change file (`valid-vault-change`, version 1): header fields `orgId`, `vaultId`, `generation`, `deviceId`, `seq` (the device's own counter, up by 1 per file), and `iv`; then the ciphertext; then `sig`, the device's ECDSA signature over the header and ciphertext. Inside the ciphertext is a list of entry operations: `entryId`, `counter`, `deviceId`, `op` (`put` or `delete`), and the entry data.

Readable without a key: ids, counters, file sizes, and timing. Entry names, usernames, URLs, notes, and passwords are all encrypted.

Settings file (`valid-vault-settings`, version 1): `orgId`, `epoch`, the company public key, the recovery public key, executive public keys and threshold, the list of protected actions, and policy values (storage mode, personal vaults, wipe count, auto-lock, key export, QR key sharing, vault export). Signed by the company key.

Device code: a QR code carrying the device's public keys, plus 6 BIP-39 words derived from a SHA-256 hash of them, which the admin and employee compare.

Re-key journal: kept on the admin device, encrypted under the admin's master key: operation id, vault id, old and new generation, current state, the per-device last sequence number included in the snapshot, and the expected fingerprint, a SHA-256 over the

canonical decrypted entries.

Recovery piece: a SLIP-0039 word share, printed and shown as a QR code.

6. Cryptography

Operation	Algorithm and parameters	Context label
Wrap a vault key to a device	Fresh P-256 ECDH key pair per wrap, ECDH with the device's public key, HKDF-SHA256 with an empty salt and the label as info, AES-256-GCM with a random 12-byte IV	`valid-vault/wrap/v1/<orgId>/<vaultId>/<generation>/<deviceId>`
Encrypt a change file	AES-256-GCM under the vault key, random 12-byte IV	`valid-vault/change/v1/<orgId>/<vaultId>/<generation>/<deviceId>/<seq>`
Encrypt a snapshot	AES-256-GCM under the vault key, random 12-byte IV	`valid-vault/snapshot/v1/<orgId>/<vaultId>/<generation>`
Sign	ECDSA P-256 with SHA-256, raw 64-byte signature	Covers the canonical bytes
Store device private keys	AES-256-GCM under the device master key, through the existing `encrypt`	`valid-vault/device-key/v1`

Operation	Algorithm and parameters	Context label
Recovery key	A P-256 ECDH key pair. Its 32-byte private key is split with SLIP-0039. Vault keys wrap to its public key, so re-keys never need the pieces	`valid-vault/wrap/v1/<orgId>/<vaultId>/<generation>/recovery`

Public keys must be 65-byte uncompressed P-256 points and are imported through WebCrypto, which rejects points off the curve. Each re-key starts a new vault key, which keeps random IVs far below the collision limit for one key.

7. Processing and authorization

A device processes a shared vault in this order, and any failure skips that file without applying any part of it:

1. Verify the settings file against the pinned company key. Refuse an epoch lower than the highest seen.
2. Verify the manifest's company signature, plus executive approvals where the action needs them. Refuse a lower epoch, or the same epoch with different contents.
3. Find this device's wrap. If it is missing, the device has been removed: stop syncing and say so.
4. Unwrap the vault key.
5. For each change file: check size limits, format, and version; check the writer is a `writer` in the manifest for that generation; verify the device signature; refuse a `seq` already seen; decrypt with the context label; validate every entry; then merge.

Change files can only carry vault entries. Membership, settings, and protected actions only change through files the company key signs. The full rules table is in the Valid Vault Enterprise Security Design, section 6.

8. Merge rules

- Every entry carries a `counter` and the `deviceId` that last wrote it.

- An edit sets `counter` to one more than the highest counter the device has seen for that entry.
- The higher counter wins. A tie goes to the lower `deviceId`, so every device picks the same winner.
- A delete is a tombstone with its own counter and follows the same rule.
- Tombstones are kept until the next re-key snapshot, then dropped.
- No device clock ever decides the order.

9. Re-key state machine

State	What is done	If interrupted here
1. Started	Journal written: vault, old and new generation	Next run deletes any partial `gen-<n+1>` folder and starts over
2. Key made	New vault key made, wraps for remaining devices and recovery stored in the journal	Start over from state 2
3. Snapshot written	Full snapshot under the new key written to `gen-<n+1>`	Read it back and verify, or delete and rewrite it
4. Verified	Snapshot read back, fingerprint matches the journal	Continue to publish
5. Published	Signed manifest at the next epoch written. This is the single commit point	Continue to clean up
6. Retired	Old-generation changes written after the snapshot are re-encrypted into the new generation, except the removed device's. The old folder is deleted after the company's retention period	Done

Before state 5, nothing members can see has changed, so roll back means deleting the new folder. After state 5 the process only moves forward.

10. Stages and tasks

Each stage is delivered, reviewed, and pushed before the next starts. A stage is done when its exit tests pass, and every test is checked by breaking the code it covers and confirming it fails.

Stage 0: Feasibility (2 weeks, needs nothing)

Proves the extension can run a shared folder before any format work starts.

- Pick a folder from `manage.html` with `showDirectoryPicker` and keep the handle in IndexedDB, as `linkedvault.js` does for one file
- List, read, create, overwrite, and delete files on a mapped drive, a UNC path, and a NAS share
- Two machines writing different files in one folder at once, and one reading while another writes
- Kill the browser mid-write and record what is left: the old file, a partial file, or a swap file
- Check whether access survives a browser restart or needs a click each session, and whether it works from `background.js` or an offscreen document
- Repeat on Edge, and on Brave with its file access flag
- Time listing and reading 1,000 small files over SMB

Exit: every operation works on Chrome and Edge over a mapped drive and a NAS, with results written up in the repo. If any fails, a fallback is chosen and this plan is updated before stage 1.

Stage 1: Crypto foundation (3 weeks, needs 0)

- Canonical encoding (section 5)
- ECDH wrap and unwrap with HKDF context binding, ECDSA sign and verify, public key checks, in `orgcrypto.js`
- SLIP-0039 split and combine in `recoverysplit.js`
- Device private keys stored under the master key
- Measure the key backup file's guess rate with hashcat mode 10900 and update the Security Design

Exit: fixed test vectors for wrap and sign pass, the published SLIP-0039 vectors pass, any changed byte is refused, and 10,000 fuzzed inputs are refused cleanly.

Stage 2: Company identity and settings (2 weeks, needs 1)

- Create a company: signing key, recovery key pair, split, pieces printed and shown as QR
- Admin's backup of the company signing key in the `keypackage.js` format
- Build, sign, and verify the settings file
- First import pins the company key after the short code check
- Delivery through a Chrome managed storage schema and Android managed configuration
- First rules enforced: auto-lock, key export, QR key sharing, vault export

Exit: unsigned, forged, wrong-company, and older settings are refused, and each rule is enforced in the extension and the app.

Stage 3: Enrollment and authorization (3 weeks, needs 2)

- Device key pairs and the device code, QR plus 6 words
- Admin approval writes a new manifest at the next epoch
- Reader and writer roles
- Executive keys, threshold, and the protected-action list
- Epoch tracking, with rollback and replay refused

Exit: mismatched words refuse the add, a manifest signed by a member is refused, a protected action without enough executive signatures is refused, and a lower or duplicate epoch is refused.

Stage 4: Shared vault storage (5 weeks, needs 3)

- `sharedfolder.js` finished from the stage 0 prototype
- Manifest and change file writers
- The processing order from section 7, with size limits
- Merge by edit counter with tombstones (section 8)
- A background sync loop that polls the folder at an interval set in company settings
- Shared entries offered for fill next to personal ones, labeled with the vault name

Exit: two profiles and two machines edit at once, both edits survive, and every copy ends with the same fingerprint. 10,000 damaged and altered files are refused without a crash. IVs never repeat across writers in a test of 1 million encryptions. An unknown version stops.

Stage 5: Removal, re-keying, and recovery (4 weeks, needs 4)

- `rekey.js` with the six states from section 9
- Old-generation changes folded into the new generation, the removed device's excluded
- Retirement after the retention period
- Offline revocation: a reconnecting removed device stops syncing
- Recovery: combine pieces, unwrap, enroll a new admin device, and replace the company signing key, with executive approval where the company requires it
- Recovery drill: rebuild the private key, derive its public key, and compare it to the recovery public key in settings, without opening any vault

Exit: a removed device reads nothing saved after removal. A re-key killed at each of the six states recovers. Any 3 of 5 pieces work, 2 do not, and a wrong piece is caught by its checksum.

Stage 6: Organization tab (4 weeks, needs 5)

- Admin screens: company setup, devices (pending, approve, remove), vaults and members, executives and protected actions, recovery drill, settings
- Employee screens: join a company, show the device code, status of each shared vault
- Plain messages for every failure: removed, settings refused, folder unreachable, permission needed
- Admin section in `guide.html`

Exit: a Playwright flow for every screen, and a test for every failure message.

Stage 7: Phones and deployment (3 weeks, needs 6)

- Phone enrollment, with device keys protected by the Android Keystore unlock
- Desktop sends the shared snapshot to the phone by QR fountain. The phone reads and fills
- Android managed configuration
- Deployment package: Chrome policy JSON, Brave flag steps, BitLocker and Offline Files checklist

Exit: the phone opens only its own vaults and cannot edit, and the checklist runs end to end on a clean Windows machine and a NAS.

Stage 8: Security review and pilot (4 weeks, needs 7)

- Full test suite on every push
- Outside security review of the formats and cryptography
- Fix findings
- Run the pilot (section 13)

Exit: every high-severity review finding is fixed and the pilot pass criteria are met.

11. Verification

Everything below runs on every push next to the existing 193 tests and 27 Android unit tests, except the checks marked by hand, whose results are recorded in the repo.

Check	How it runs	What it proves	Stage
Shared folder operations	By hand on two machines, a mapped drive, and a NAS	The browser can run a shared folder	0, 4, 7
Crypto vectors	Node test runner with fixed vectors and published SLIP-0039 vectors	Wraps, signatures, and pieces are correct	1
Parser fuzzing	10,000 damaged settings files, manifests, change files, device codes, and pieces	Bad input is refused and never touches a vault	1 to 4
Authorization	Playwright with forged, member-signed, and replayed files	Only the company key changes membership and settings	3
Two writers	Two browser profiles, then two machines, on one folder	Simultaneous edits converge	4
IV uniqueness	1 million encryptions across simulated writers	No IV repeats under one key	4
Interrupted re-key	Process killed at each of six states, then resumed	Finish or roll back, then verify	5
Removed device	A removed profile with a stale local copy	Reads nothing saved after removal	5

Check	How it runs	What it proves	Stage
Phone delivery	Phone test harness	Opens only its own vaults, cannot edit	7
Mutation checks	Each fix broken on purpose	Every test fails without the code it covers	All
Outside review	Independent reviewer	Formats and crypto hold up	8

12. Platforms

Platform	Phase 1 support	Notes
Chrome on Windows	Full	File System Access API in extension pages
Edge on Windows	Full	Same engine, confirmed in stage 0
Brave on Windows	Full, with its file access flag on	Flag steps in the deployment package
Android	Read and fill	QR delivery and managed configuration
Windows file server or NAS over SMB	Storage	Mapped drive or UNC path, confirmed in stage 0
macOS, Linux, Firefox, Safari	Not in phase 1	Not part of the pilot

13. Pilot

Site: the first pilot business, to be named. 3 to 10 people, at least 2 shared vaults, at least 1 phone, and a Windows file server or NAS.

Setup: run the deployment checklist, create the company, enroll every device, and move existing logins in through the import flow.

Run: 4 weeks of daily use. During it, run these drills:

- Remove a device and confirm the re-key

- Recover admin access with the pieces on a test vault
- Restore the shared folder from backup
- Two people edit the same entry at once
- A laptop stays offline for a day, then reconnects

Pass criteria:

- No lost or damaged entries, with fingerprints matching across devices every day
- Every drill passes
- No shared vault data left on network-only laptops after removal
- Employees fill logins without help after the first week
- No high-severity issue open at the end

Rollback: personal vaults are never touched. The admin exports each shared vault to an encrypted vault file, people return to their previous method, and the shared folder is deleted.

14. Schedule

Stage	Estimate	Needs
0. Feasibility	2 weeks	Nothing
1. Crypto foundation	3 weeks	0
2. Company identity and settings	2 weeks	1
3. Enrollment and authorization	3 weeks	2
4. Shared vault storage	5 weeks	3
5. Removal, re-keying, recovery	4 weeks	4
6. Organization tab	4 weeks	5
7. Phones and deployment	3 weeks	6
8. Security review and pilot	4 weeks	7
Total, plus 20% contingency	36 weeks	

Assumptions: one full-time engineer for all of it, the outside review booked to start during stage 7, and every estimate revised at the end of stage 0. Each stage ships as its own release, with enterprise features hidden behind a setting until stage 8 passes.

15. Decisions

Ticked decisions are settled. Each names the stage that needs it settled.

- [x] ~~**Each device gets a signing key** and signs every change file it writes, so each change traces to a device. Stage 1~~
- [x] ~~**Each company picks which actions need several executives,** set in its signed settings, with each executive holding their own signing key. Stage 3~~
- [x] ~~**The recovery split is written in-house** and tested against published test vectors, keeping the no-new-dependencies rule. Stage 1~~
- [x] ~~**The recovery split follows SLIP-0039,** so its published test vectors and word format apply directly. Stage 1~~
- [ ] **Merge order in shared vaults uses edit counters.** A coworker's wrong clock could otherwise win every merge. Personal vaults keep timestamps. Stage 4
- [ ] **Phones read and fill shared vaults, and edits happen on a desktop.** Phone editing comes later. Stage 7
- [ ] **The first settings import pins the company key after a short code check,** compared in person or by phone. Stage 2
- [ ] **Admin tools go in a new Organization tab in Manage.** Stage 6
- [ ] **The recovery split is chosen per company at setup,** with 3 of 5 as the default. Stage 2
- [ ] **Brave keeps the documented file access flag,** the same as Uni-Vault. Stage 0
- [ ] **Local-copy devices keep their last shared snapshot offline** until they reconnect, then stop syncing. Network-only devices are cut off at once. Stage 5
- [x] ~~**Old-generation files are deleted after a retention period the company sets,** default 30 days. This clears the share and does not reach copies a device already holds. Stage 5~~
- [ ] **Fill-only mode in phase 2 turns off developer tools by Chrome policy** as one layer, with its own security review. Phase 2
- [ ] **Release numbering for phase 1,** for example 0.10.0 when stage 1 ships. Stage 1
- [ ] **Outside reviewer and budget** for the stage 8 review. Stage 7

Company deployment options, set per company in the signed settings: shared vault storage (local copy or network only), personal vaults (folder on the company drive or none on company laptops), wipe after a set number of failed unlocks, and BitLocker on the deployment checklist.